MARKETIX

An Intelligent Marketing Analytics and Customer Insights Platform Using Python and Machine Learning

Student: Kamandeep Singh | **Academic Level:** B.Tech CSE 3rd Year (Industrial Training Project)
Certification Integrations: BCG X (Decision Analytics), IBM (Data Analysis with Python), Deloitte (Job Simulation)
Generated: November 08, 2026 | **Platform Architecture:** Decoupled Layered Architecture (Streamlit + PostgreSQL + Scikit-Learn)

---

1. Executive Summary & Problem Context

Modern businesses face high customer acquisition costs (CAC). Organizations frequently broadcast promotional offers across email, SMS, and direct mail indiscriminately. In typical commercial retail datasets, **only ~14.9% of prospects convert**, meaning 85% of marketing expenditure is wasted on uninterested prospects. **MARKETIX** solves this problem through an end-to-end intelligence engine that unifies data cleaning, unsupervised persona discovery, and supervised response propensity scoring into a single interactive platform.

Layer	Technology Stack	Core Industrial Responsibility
1. Presentation	Streamlit, HTML5, CSS3 Animations	8 modular interactive web views, KPI cards, real-time filters, Plotly dashboards.
2. Application	Python 3.13, Pandas, NumPy	Data ingestion, validation, median imputation, and domain feature engineering.
3. Database	PostgreSQL 16 & SQLite Fallback	6 relational tables, transactional sync, foreign keys, and offline resilience.
4. Machine Learning	Scikit-Learn, Joblib	Unsupervised K-Means clustering (K=4) + Supervised balanced Logistic Regression (AUC 0.901).
5. Output / Delivery	ReportLab, Plotly, CSV	Executive PDF briefs, Plotly charts, prioritized consulting action steps, CSV exports.

2. Function-by-Function Technical Reference

Comprehensive documentation of every core function across the codebase:

Module / File	Function Name & Signature	Inputs & Logic	Return Value & Output
src/data_loader.py	load_dataset(file_or_path)	Detects comma, semicolon, tab or Excel formats.	Tuple: (DataFrame, status_msg)
src/data_loader.py	get_dataset_summary(df)	Computes rows, columns, nulls, duplicates, and MBs.	Dict: data health metrics
src/data_cleaning.py	clean_marketing_data(df)	Imputes Income with education median; caps ages > 100; normalizes status; creates Total_Spending & Tenure.	Tuple: (Cleaned_DF, report_dict)
src/analytics.py	compute_kpis(df)	Calculates Customers, Campaigns, Responses, Response Rate, Avg Income, Avg Spending.	Dict: executive dashboard KPIs
src/analytics.py	compute_category_spend_breakdown(df)	Aggregates spend across Wines, Meat, Gold, Fish, Sweets, and Fruits.	DataFrame: category revenue share
src/analytics.py	compute_purchase_channel_breakdown(df)	Sums purchases across Store, Web, Catalog, Deals.	DataFrame: channel volume & shares
src/campaign_analysis.py	analyze_campaign_performance(df)	Benchmarks Campaign 1..5 vs Target Campaign; flags top-converting wave; counts acceptance multiplicity.	Dict: summary_table, best_campaign, engagement_dist
src/campaign_analysis.py	analyze_channel_performance(df)	Analyzes customer transactions across channels.	DataFrame: channel stats
src/customer_analysis.py	analyze_demographics(df)	Groups age brackets (<35, 35-50, 50-65, 65+); education counts; marital status breakdown.	Dict: demographic distributions
src/customer_analysis.py	analyze_spending_by_tier(df)	Splits Low, Moderate, High spenders; checks conversion rate across tiers (proves 3x lift).	DataFrame: tier conversion rates

src/customer_analysis.py	analyze_product_affinity(df)	Compares average category spend between campaign responders and non-responders.	DataFrame: spend premium (%)
ml/segmentation.py	calculate_elbow_and_silhouette(df, max_k)	Executes K-Means for K=2..7; calculates WCSS inertia and silhouette scores.	Dict: inertias, silhouettes, optimal_k
ml/segmentation.py	perform_kmeans_clustering(df, n_clusters)	StandardScaler normalizes features; fits K-Means; automatically assigns business personas.	Tuple: (DF_with_clusters, profile_df, metrics)
ml/train_model.py	train_logistic_regression(df)	Prevents data leakage; stratified 80/20 split; class_weight='balanced'; evaluates AUC, Recall, F1; persists joblib.	Dict: metrics, confusion_matrix, roc_curve, coefficients
ml/predict.py	predict_single_customer(data, threshold)	Encodes single customer vector; transforms with scaler; evaluates probability.	Dict: positive/negative probabilities, prediction label
ml/predict.py	predict_batch_customers(df, threshold)	Scores all 2,240 records; appends Predicted_Probability and Response_Likelihood.	DataFrame with appended prediction columns
database/connection.py	test_connection()	Tests PostgreSQL at localhost:5432; on failure silently routes to SQLite marketix.db.	SQLAlchemy Engine instance
database/queries.py	sync_processed_data_to_db(df)	Synchronizes customers, campaigns definitions, and responses into relational tables.	Tuple: (bool_success, msg)
reports/generator.py	generate_pdf_report(metrics, cmp, recs)	Builds high-resolution vector PDF in memory with executive KPI and recommendation tables.	bytes: downloadable PDF buffer

3. Step-by-Step Usage Guide with Concrete Examples

Example 1: Ingesting & Cleaning Real Marketing Data

Workflow: Navigate to *Module 2: Data Management*. Upload a marketing CSV or click the default file.
• **Input Data:** Raw CSV with 2,240 records, 24 missing values in Income, anomalous marital statuses ('YOLO', 'Absurd'), and birth years in 1893.
• **Action:** Click *'■ Execute Data Cleaning & Sync to Database'*.
• **Outcome:** Imputes missing incomes based on Education tier median (e.g. Master median = $50,943), normalizes marital status to 'Single', caps age outliers at the cohort median (56 yrs), and adds engineered metrics (Total_Spending, Customer_Tenure_Days). Records are written to database tables.

Example 2: Benchmarking Multi-Wave Campaigns

Workflow: Navigate to *Module 3: Campaign Performance Analysis*.
• **Data:** Multi-wave campaigns (Campaign 1: 6.43%, Campaign 2: 1.34%, Campaign 3: 7.28%, Campaign 4: 7.46%, Campaign 5: 7.28%, Latest Campaign: 14.91%).
• **Business Finding:** The Latest Campaign converted at **14.91% (334 responses)**, outperforming Campaign 2 by more than 11x.
• **Channel Conversion:** Reveals that physical store purchases drive total sales volume (5,800+ transactions), but direct catalog mailings produce higher conversion efficiency among affluent households.

Example 3: Discovering Customer Personas (K-Means)

Workflow: Navigate to *Module 5: Customer Segmentation*.
• **Diagnostics:** Inspect the Elbow Curve and Silhouette score graph (validating K=4 as optimal).
• **Personas Discovered:**
1. **Affluent Champions:** Avg Income $75,000+, Total Spend $1,200+, high Wine & Meat spend, 28% campaign response rate.
2. **Budget Seekers:** Avg Income <$35,000, Total Spend <$100, deal-driven, low campaign acceptance (<8%).
3. **Cautious Savers:** Good income ($55k+), moderate spending, requires educational drip email nurturing.
4. **High-Value Enthusiasts:** Frequent website visitors, high web catalog purchases, responsive to multi-channel promotions.

Example 4: Scoring Prospect Conversion Probability (Logistic Regression)

Workflow: Navigate to *Module 6: Response Prediction*.
• **Case A (High-Intent Prospect):** Enter Customer ID 105, Age 45, Income $75,000, Wine Spend $500, Meat Spend $350, Accepted Campaign 1 & 5. Click *'Predict'*.
→ **Result: 78.2% Positive Response Probability**. System outputs *'■ LIKELY TO RESPOND'* and recommends including in high-touch VIP mailings.
• **Case B (Low-Intent Prospect):** Enter Customer ID 210, Income $24,000, Wine Spend $10, No previous campaigns accepted.
→ **Result: 8.4% Probability**. System outputs *'■ UNLIKELY TO RESPOND'* and advises excluding from high-cost direct print mailings to eliminate ad waste.

Example 5: Automated Consulting Recommendations & PDF Brief Export

Workflow: Navigate to *Module 7: Recommendations* and *Module 8: Reports*.
• **Strategy Cards:** Generates prioritized action items adhering to BCG X / Deloitte frameworks, explicitly separating correlation from causation.
• **One-Click Export:** Compiles vector PDF executive reports via ReportLab and generates CSV exports of segmented and scored customers.

4. Faculty Presentation Guide: 7-Minute Viva Script

Deliver this exact structured presentation during your B.Tech 3rd-year industrial training viva:

Time & Phase	Screen to Show	Verbatim Viva Presenter Script
0:00 - 1:00 Introduction	Home Portal (app.py)	*'Good morning, respected professors and examiner. Today, I am presenting **MARKETIX**, an industrial-grade marketing intelligence platform developed using Python and Machine Learning. This project consolidates my industrial training with certifications from **BCG X**, **IBM**, and **Deloitte**. The core objective is solving marketing budget wastage: in typical retail datasets, only ~15% of prospects respond, while 85% of ad spend is wasted. MARKETIX provides data cleaning, customer segmentation, and response prediction.'*

1:00 - 2:15 Architecture & Data Cleaning	Module 2: Data Management	*'Here in Module 2, our application ingests marketing data and performs automated data cleaning. The raw dataset has 2,240 records with 24 missing incomes. Rather than deleting rows, we implement **Education-stratified median imputation** to preserve demographic fidelity. We also cap birth-year outliers and engineer marketing KPIs like Total Spending and Customer Tenure. The cleaned records synchronize into our 6-table PostgreSQL schema, with resilient SQLite fallback.'*
2:15 - 3:30 Dashboard & Campaign Analysis	Module 1 & 3: Dashboard & Campaigns	*'Looking at our executive dashboard, we see 2,240 customers and 334 conversions on the target campaign (14.91% acceptance rate). In Module 3, our multi-wave campaign analysis reveals that the latest integrated campaign outperformed Campaign 2 by more than 11x. We also analyze omnichannel distribution, showing physical store transactions lead volume while direct catalog mailings produce higher conversions among affluent households.'*
3:30 - 4:45 Customer Segmentation (K-Means)	Module 5: Segmentation	*'In Module 5, we apply Unsupervised Machine Learning. To avoid arbitrary clustering, we execute an automated **Elbow Method** and **Silhouette Score** diagnostic for K=2 to 6, confirming K=4 as optimal. Our algorithm identifies 4 actionable personas: **Affluent Champions** (high income, high spend, 28% response rate), **Budget Seekers** (deal hunters), **Cautious Savers**, and **High-Value Enthusiasts**. Each persona receives distinct marketing recommendations.'*
4:45 - 6:00 Response Prediction (Logistic Regression)	Module 6: Prediction & ROC	*'In Module 6, we predict customer response likelihood using Supervised Logistic Regression. We strictly prevent data leakage and handle class imbalance using class_weight="balanced" on a stratified 80/20 train/test split. Our model achieves an **ROC-AUC of 0.901** and a **Recall of 77.6%** on positive responders. As demonstrated in this interactive form, entering customer ID 105 computes a 78.2% conversion likelihood in real time, validating high-touch targeting.'*
6:00 - 7:00 Recommendations, Reports & Conclusion	Module 7 & 8: Recommendations & PDF	*'Finally, in Module 7 and 8, our platform translates analytical findings into prioritized consulting initiatives adhering to BCG X / Deloitte frameworks, explicitly distinguishing correlation from causation. Stakeholders can compile and download an executive vector PDF brief generated via ReportLab or export CSV datasets with one click. MARKETIX bridges machine learning theory with industrial business execution. Thank you, and I welcome your questions.'*

5. Top 10 Viva Defense Questions & Examiner Answers

Q1: Why did you choose Streamlit instead of React and FastAPI?

Model Answer: In marketing analytics, unified Python execution eliminates serialization latency, boilerplate REST controllers, and CORS overhead. Streamlit binds directly with Pandas, Scikit-Learn, and SQLAlchemy, delivering reactive dashboards without compromising machine learning speed or code maintainability.

Q2: How did you handle class imbalance in the response prediction task?

Model Answer: Because only 14.9% of prospects responded, an unweighted model could achieve 85% accuracy by predicting zeros for every prospect, rendering it commercially useless. We solved this by using a stratified 80/20 train/test split and configuring class_weight='balanced' in Logistic Regression, penalizing minority false negatives and achieving 77.6% Recall and 0.901 ROC-AUC.

Q3: Why not delete records with missing Income values?

Model Answer: Deleting rows with missing data reduces sample size and introduces systematic demographic bias. Instead of simple mean imputation (which distorts variance), we implemented Education-stratified median imputation: imputing each missing income using the median income of that customer's specific educational degree.

Q4: How do you mathematically justify K=4 in K-Means clustering?

Model Answer: Rather than picking an arbitrary number of clusters, our platform executes an automated Elbow Curve (within-cluster sum of squares inertia) and calculates Silhouette Scores across K=2 to 6. K=4 demonstrates the optimal balance of high intra-cluster cohesion and inter-cluster separation.

Q5: Why choose Logistic Regression over complex black-box models like Random Forest or XGBoost?

Model Answer: For a B.Tech 3rd-year industrial training project, explainability and odds-ratio interpretability are paramount. Logistic Regression provides explicit linear coefficients that explain why a customer is predicted to convert, runs with low latency, and achieves an outstanding ROC-AUC of 0.901 without overfitting.

Q6: How does the database layer maintain reliability if PostgreSQL is offline?

Model Answer: Our database manager in database/connection.py implements a dual-mode pattern. It tests connection to PostgreSQL at localhost:5432; if the PostgreSQL daemon is offline, it activates an embedded SQLite database (marketix.db) without throwing crashes, ensuring reliable viva demonstrations.

Q7: How did you prevent data leakage during model training?

Model Answer: We isolated the target response column (Response) and excluded non-predictive attributes (ID) as well as post-campaign indicators. Features are scaled using parameters fitted strictly on the 80% training split, preventing test data leakage into training transformations.

Q8: Why does MARKETIX avoid claiming an explicit ROI calculation?

Model Answer: The Kaggle dataset contains placeholder columns (Z_CostContact=3, Z_Revenue=11) rather than true variable marketing acquisition costs and product gross margins. Following ethical consulting standards (emphasized in BCG X), we benchmark empirical Response Rates and Category Spend Lift rather than computing speculative ROI.

Q9: What does the BCG X and Deloitte connection contribute to this project?

Model Answer: BCG X principles are applied in Modules 3 and 7: translating data patterns into decision frameworks, managing customer lifetime value, and separating correlation from causation. Deloitte simulation practices are implemented in Modules 5, 6, and 8: behavioral segmentation, predictive scoring, and executive PDF briefs.

Q10: What are the primary future enhancements for MARKETIX?

Model Answer: Future iterations can integrate multi-armed bandit algorithms for automated A/B test traffic allocation, containerized cloud deployment on AWS/GCP via Docker, and real-time Kafka event streaming for live e-commerce website browse tracking.